21

超越物理内存：机制

到目前为止，我们一直假设地址空间（address space）小得不切实际，能放进物理内存（physical memory）。实际上，我们一直假设所有运行中进程（process）的地址空间全都能放进内存。现在我们要放宽这些很强的假设，转而支持许多同时运行的大地址空间。

为此，我们需要在**存储器层次结构**（memory hierarchy）中再加一层。到目前为止，我们一直假设所有页（page）都驻留在物理内存中。然而，为了支持大地址空间，操作系统需要一个地方，把地址空间中当前不太急需的部分暂时存放起来。一般来说，这个地方的特点是容量要比内存大；因此它通常也更慢（要是它更快，我们直接把它当内存用不就行了?）。在现代系统中，这个角色通常由**硬盘驱动器**（hard disk drive）来担任。于是，在我们的存储器层次结构中，又大又慢的硬盘位于最底层，内存就在它上面一层。由此就引出了问题的关键：

关键问题：如何超越物理内存
操作系统怎样利用一个更大、更慢的设备，透明地提供巨大虚拟地址空间（virtual address space）的假象?

你可能会问：为什么要为一个进程支持单个巨大的地址空间? 答案仍然是方便和易用。有了巨大的地址空间，你就不必操心内存里有没有足够的空间放程序的数据结构（data structure）；只要自然地写程序，按需分配内存就行。这是操作系统提供的一个强大假象，让你的生活简单了很多。不用谢！与之形成对比的是使用**内存覆盖**（memory overlays）技术的老系统，这种技术要求程序员根据需要，手动把一段段代码或数据移进移出内存 [D97]。想象一下那会是什么样子：调用函数或访问某些数据之前，你得先设法把这段代码或数据放进内存；真够呛！

有了交换空间（swap space），操作系统不仅能为单个进程，还能为多个同时运行的进程提供巨大虚拟内存的假象。多道程序设计（multiprogramming，即“同时”运行多个程序，以更好地利用机器）发明以后，几乎就必须能把一部分页换出（swap out），因为早期的机器显然无法同时容纳所有进程所需的全部页。因此，多道程序设计加上对易用性的追求，让我们希望能使用比实际物理内存更多的内存。这是所有现代虚拟内存系统都会做的事；接下来我们就深入了解它。

补充：存储技术

后面我们会更深入地探讨 I/O 设备实际上是如何工作的（见 I/O 设备一章），所以请耐心一点！当然，这个较慢的设备不一定是硬盘，也可以是更现代的设备，比如基于闪存（flash）的 SSD。这些我们也会讨论。现在，只要假设我们有一个容量大、速度相对较慢的设备，可以借助它营造出一个非常大的虚拟内存（virtual memory）假象，这个虚拟内存甚至比物理内存本身还大。

21.1 交换空间

我们要做的第一件事，是在磁盘上预留一些空间，用来把页移来移去。在操作系统中，我们一般把这样的空间称为**交换空间**（swap space），因为我们把内存中的页换出（swap out）到这里，也从这里把页换入（swap in）内存。我们就简单地假设，操作系统能以页为单位读写交换空间。为此，操作系统需要记住某一页的**磁盘地址**（disk address）。

交换空间的大小很重要，因为它最终决定了系统在某一时刻最多能使用多少个内存页（memory page）。为简单起见，我们暂时假设它非常大。

图 21.1 是一个小例子，其中有一个 4 页的物理内存和一个 8 页的交换空间。进程 0、1、2 这三个进程正在共享物理内存；不过，这三个进程都只有一部分有效页（valid page）在内存中，其余的页都在磁盘上的交换空间里。第四个进程（进程 3）的所有页都被换出到了磁盘上，所以它显然当前没有运行。交换空间中还有一个块（block）是空闲的。哪怕只是这样一个小例子，也希望你能从中看出，系统是如何利用交换空间假装内存比实际更大的。

需要注意的是，交换空间并不是磁盘上唯一用于交换的位置。例如，假设你在运行一个程序的二进制文件（比如 `ls`，或者你自己编译的 `main` 程序）。这个二进制文件的代码页（code page）最初在磁盘上，程序运行时，它们会被加载到内存中（要么在程序开始执行时一次性全部加载，要么像现代系统那样，在需要时一次加载一页）。不过，如果系统需要为其他用途腾出物理内存，它可以放心地重新使用这些代码页占用的内存空间，因为以后随时可以从文件系统（file system）里那个二进制文件中把它们重新换入。

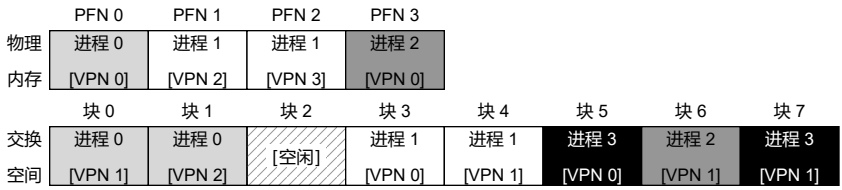


图 21.1：**物理内存和交换空间**

21.2 存在位

现在磁盘上已经有了一些空间，我们需要在系统的更高层添加一些机制，以便把页换出到磁盘，或从磁盘换入。为简单起见，我们假设系统使用由硬件管理的 TLB（hardware-managed TLB）。

首先回想一下引用内存时会发生什么。运行中的进程会产生虚拟内存引用（取指令或访问数据），在我们假设的这种系统中，硬件会先把它们转换为物理地址（physical address），再从内存中取出所需的数据。

记住，硬件先从虚拟地址（virtual address）中提取 VPN，然后检查 TLB 中有没有匹配项（即 **TLB 命中**，TLB hit）；如果命中，就生成相应的物理地址，并从内存中取数据。我们希望这是常见情况（common case），因为它很快（不需要额外的内存访问）。

如果 TLB 中找不到这个 VPN（即 **TLB 未命中**，TLB miss），硬件就会（借助**页表基址寄存器**，page table base register）在内存中找到页表（page table），并用 VPN 作为索引，查找这一页的**页表项**（page table entry，PTE）。如果这一页有效，并且就在物理内存中，硬件就从 PTE 中取出 PFN，把它装入 TLB，然后重试（retry）这条指令，这次就会产生 TLB 命中；到这里一切顺利。

但是，如果我们希望允许页被交换到磁盘上，就得再加一些机制。具体来说，硬件查看 PTE 时，可能会发现这一页不在物理内存中。硬件（在软件管理 TLB 的方案中则是操作系统）要借助每个页表项中的一项新信息来判断这一点，这项信息叫作**存在位**（present bit）。如果存在位（present bit）为 1，就表示这一页在物理内存中，一切都像上面说的那样进行；如果为 0，这一页就不在内存中，而是在磁盘上的某个地方。

访问不在物理内存中的页，这一行为通常称为**缺页异常**（page fault）。

发生缺页异常时，操作系统会出面：一段专门的代码，即**缺页异常处理程序**（page-fault handler），开始运行并处理这次异常。下面我们就来介绍它。

补充：交换相关的术语及其他

虚拟内存系统中的术语可能有点让人困惑，而且因机器和操作系统而异。例如，从更宽泛的意义上说，**缺页异常**（page fault，也叫缺页中断）可以指任何引发了某种故障的页表引用：既可能是我们这里讨论的这类故障，即页不存在故障，有时也可能是非法内存访问。事实上，一次访问明明完全合法（访问的页已经映射到进程的虚拟地址空间中，只是当时不在物理内存里），却被称为“故障”（fault），这本身就有点奇怪；它其实应该叫**缺页**（page miss）。不过，人们说某个程序“频繁缺页”时，通常是指它在访问自己虚拟地址空间中被操作系统换出到磁盘的那些部分。

我们猜想，这种行为之所以被称为“故障”，与操作系统中处理它的机制有关。一旦发生了不寻常的事情，也就是硬件不知道该如何处理的事情，硬件就干脆把控制权交给操作系统，指望它能把事情处理好。在这种情况下，进程想访问的页不在内存中；硬件只能做它唯一能做的事：引发异常，然后由操作系统接手。由于这和进程做了非法操作时发生的事情完全一样，把这种情况称为“故障”，也许就不足为奇了。

补充：为什么硬件不处理缺页异常

从 TLB 的经验我们知道，硬件设计者几乎什么事都不愿意交给操作系统去做。那他们为什么放心让操作系统来处理缺页异常呢？主要原因有几个。第一，需要访问磁盘的缺页异常很慢；即使操作系统处理一次异常要花很长时间、执行大量指令，磁盘操作本身向来要慢得多，相比之下，运行软件的额外开销微乎其微。第二，为了处理缺页异常，硬件就得懂交换空间、懂怎样向磁盘发出 I/O，还得知道许多它目前不太清楚的细节。因此，出于性能和简单性两方面的考虑，缺页异常由操作系统来处理，连搞硬件的人也会满意。

21.3 缺页异常

回想一下，在 TLB 未命中的处理上，系统分为两类：一类是硬件管理的 TLB（hardware-managed TLB），由硬件在页表中查找所需的转换；另一类是软件管理的 TLB（software-managed TLB），由操作系统查找。无论哪类系统，只要页不存在，都由操作系统负责处理缺页异常。这时会运行操作系统的**缺页异常处理程序**（page-fault handler，这个名字起得恰如其分），由它决定该怎么做。几乎所有系统都用软件处理缺页异常；即使 TLB 由硬件管理，硬件也会把这项重要职责交给操作系统。

如果页不存在，并且已经被交换到磁盘上，操作系统就需要把这一页换入内存，才能处理缺页异常。于是就有一个问题：操作系统怎么知道去哪里找所需的页？在许多系统中，页表是存放这类信息的天然位置。因此，PTE 中通常存放该页 PFN 等数据的那些位，操作系统可以拿来存放磁盘地址。操作系统收到某一页的缺页异常后，会查看 PTE 找到磁盘地址，然后向磁盘发出请求，把这一页取进内存。

磁盘 I/O 完成后，操作系统会更新页表，把这一页标记为存在，更新页表项（PTE）的 PFN 字段，记录新取进来的页在内存中的位置，然后重试这条指令。这次重试可能会产生 TLB 未命中，处理这次未命中时会用这个转换更新 TLB（也可以在处理缺页异常时就更新 TLB，省掉这一步）。最后，再重试一次时，就会在 TLB 中找到这个转换，于是从内存中转换后的物理地址处取出所需的数据或指令。

注意，在 I/O 进行期间，进程会处于**阻塞**（blocked）状态。因此，在处理缺页异常的同时，操作系统可以自由地运行其他就绪（ready）的进程。由于 I/O 代价高昂，让一个进程因缺页异常而做的 I/O 与另一个进程的执行**重叠**（overlap）起来，是多道程序系统最有效地利用硬件的又一种方式。

21.4 内存满了怎么办?

你可能已经注意到，在上面描述的过程中，我们假设有足够多的空闲内存，可以把一页从交换空间**换入**（page in）。当然，情况未必如此；内存可能已经满了（或接近满了）。因此，操作系统可能要先**换出**（page out）一页或多页，为即将调入的新页腾出空间。决定踢出、或者说**置换**（replace）哪一页的做法，称为**页面置换策略**（page-replacement policy）。

```
1 VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2 (Success, TlbEntry) = TLB_Lookup(VPN)
3 if (Success == True)   // TLB 命中
4   if (CanAccess(TlbEntry.ProtectBits) == True)
5     Offset   = VirtualAddress & OFFSET_MASK
6     PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7     Register = AccessMemory(PhysAddr)
8   else
9     RaiseException(PROTECTION_FAULT)
10 else                  // TLB 未命中
11   PTEAddr = PTBR + (VPN * sizeof(PTE))
12   PTE = AccessMemory(PTEAddr)
13   if (PTE.Valid == False)
14     RaiseException(SEGMENTATION_FAULT)
15   else
16     if (CanAccess(PTE.ProtectBits) == False)
17       RaiseException(PROTECTION_FAULT)
18     else if (PTE.Present == True)
19       // 假设 TLB 由硬件管理
20       TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
21       RetryInstruction()
22     else if (PTE.Present == False)
23       RaiseException(PAGE_FAULT)
```

图 21.2：**缺页异常控制流算法（硬件）**

实际上，人们在设计好的页面置换策略（page-replacement policy）上花了很多心思，因为一旦踢错了页，程序性能就要付出巨大代价。做出错误的决定，可能会让程序以磁盘的速度而不是内存的速度运行；以现在的技术来说，这意味着程序可能会慢 10,000 倍甚至 100,000 倍。因此，这种策略值得我们详细研究；事实上，下一章正是要做这件事。眼下只要知道有这样一种策略、它建立在本章介绍的机制之上，就够了。

21.5 缺页异常的控制流

有了这些知识，我们现在可以大致勾勒出内存访问的完整控制流了。换句话说，要是有人问你“程序从内存中取数据时会发生什么?”，你应该对各种可能的情况都心中有数。更多细节请看图 21.2 和图 21.3 中的控制流；前一张图展示硬件在地址转换（address translation）时做什么，后一张展示操作系统在发生缺页异常时做什么。

从图 21.2 的硬件控制流可以看出，发生 TLB 未命中时，现在有三种重要的情况需要理解。第一种情况是这一页既**存在**（present）又**有效**（valid），见第 18–21 行；这时，TLB 未命中处理程序（TLB miss handler）只需从 PTE 中取出 PFN，重试指令，这次就会 TLB 命中，然后像之前（多次）描述的那样继续执行。第二种情况（第 22–23 行）必须运行缺页异常处理程序（page-fault handler）：虽然这一页是进程可以合法访问的（毕竟它是有效的），

```
1 PFN = FindFreePhysicalPage()
2 if (PFN == -1)                // 没找到空闲页
3     PFN = EvictPage()         // 置换算法
4 DiskRead(PTE.DiskAddr, PFN)   // 睡眠（等待 I/O）
5 PTE.present = True            // 更新页表：
6 PTE.PFN     = PFN             // （存在位/地址转换）
7 RetryInstruction()            // 重试指令
```

图 21.3：**缺页异常控制流算法（软件）**

但它不在物理内存中。第三种（也是最后一种）情况是访问了一个无效页（invalid page），比如程序里有 bug（第 13–14 行）。这时，PTE 中的其他位都无关紧要了；硬件会捕获这次无效访问，然后运行操作系统的陷阱处理程序（trap handler），很可能会终止出错的进程。

从图 21.3 的软件控制流，可以看出操作系统处理缺页异常大致要做哪些事。首先，操作系统必须为即将调入的页找到一个物理帧（physical frame）；如果没有空闲的页，就只能等置换算法（replacement algorithm）运行，把一些页踢出内存，腾出来供这里使用。拿到物理帧之后，处理程序再发出 I/O 请求，从交换空间读入这一页。最后，等这个缓慢的操作完成，操作系统更新页表并重试指令。重试会导致 TLB 未命中，再重试一次就会 TLB 命中，这时硬件就能访问所需的内容了。

21.6 置换实际上何时发生

到目前为止，我们描述置换如何发生时，都假设操作系统会一直等到内存完全满了，才置换一页，也就是把它淘汰（evict），为其他页腾出空间。可以想象，这有点不切实际，操作系统有很多理由更主动地保留一小部分空闲内存。

为了保留少量空闲内存，大多数操作系统都会设置某种**高水位线**（high watermark，*HW*）和**低水位线**（low watermark，*LW*），帮助决定何时开始从内存中淘汰页。具体做法如下：操作系统一旦发现可用页少于 *LW* 个，就会运行一个负责释放内存的后台线程（thread）。这个线程会不断淘汰页，直到有 *HW* 个可用页为止。这个后台线程有时叫作**换页守护进程**（swap daemon 或 page daemon）[1]；它为运行中的进程和操作系统释放了一些内存，于是心满意足地去睡觉了。

一次执行多次置换，就能做一些新的性能优化。例如，许多系统会把若干页**聚集**（cluster）或**成组**（group）起来，一次性写到交换分区（swap partition），从而提高磁盘的效率 [LL82]；我们后面更详细地讨论磁盘时会看到，这种聚集能减少磁盘的寻道（seek）和旋转开销，明显提高性能。

为了配合后台换页线程，图 21.3 中的控制流需要稍作修改：算法不再直接执行置换，而只是检查有没有可用的空闲页（free page）。如果没有，就通知后台换页线程需要空闲页；后台线程释放出一些页后，会重新唤醒原来的线程，原来的线程随后就能换入所需的页，接着做自己的工作。

[1] “daemon” 这个词通常读作 “demon”，是一个古老的术语，指在后台做有用工作的线程或进程。而且（又一次！）这个术语原来也出自 Multics [CS94]。

提示：在后台做事

有工作要做时，把它放到**后台**（background）去做往往是个好主意：既能提高效率，又能把操作成组处理。操作系统经常在后台做事；例如，许多系统会先把文件写操作缓冲在内存中，然后才真正把数据写到磁盘上。这样做有很多潜在的好处：一是提高磁盘效率，因为磁盘现在可以一次收到很多写请求，从而能更好地调度它们；二是降低写操作的延迟，因为应用程序会觉得写操作很快就完成了；三是有可能减少工作量，因为这些写操作可能根本不需要写到磁盘（比如文件被删除了）；四是更好地利用**空闲时间**（idle time），因为后台工作也许可以在系统空闲时完成，从而更好地利用硬件 [G+95]。

21.7 小结

在这短短的一章中，我们引入了这样一个概念：可以访问比系统实际拥有的物理内存更多的内存。要做到这一点，页表结构就得更复杂一些，因为必须包含（某种）**存在位**（present bit），告诉我们这一页是否在内存中。如果不在，操作系统的**缺页异常处理程序**（page-fault handler）就会运行，处理这次**缺页异常**（page fault），从而安排把所需的页从磁盘传到内存，可能还要先置换内存中的一些页，为即将换入的页腾出空间。

请记住（这一点很重要，也很神奇！），所有这些操作对进程都是**透明**的。在进程看来，它只是在访问自己私有的、连续的虚拟内存。在幕后，页被放在物理内存中任意的（不连续的）位置，有时甚至根本不在内存中，需要从磁盘取回。虽然我们希望一般情况下内存访问都很快，但在某些情况下，完成一次访问需要多次磁盘操作；像执行一条指令这样简单的事情，在最坏的情况下也可能要花好几毫秒才能完成。

 © 2008–23, ARPACI-DUSSEAU

参考文献

[CS94] "Take Our Word For It" by F. Corbato, R. Steinberg. www.takeourword.com/TOW146 (Page 4). *Richard Steinberg writes: "Someone has asked me the origin of the word daemon as it applies to computing. Best I can tell based on my research, the word was first used by people on your team at Project MAC using the IBM 7094 in 1963." Professor Corbato replies: "Our use of the word daemon was inspired by the Maxwell's daemon of physics and thermodynamics (my background is in physics). Maxwell's daemon was an imaginary agent which helped sort molecules of different speeds and worked tirelessly in the background. We fancifully began to use the word daemon to describe background processes which worked tirelessly to perform system chores."*

[D97] "Before Memory Was Virtual" by Peter Denning. In the Beginning: Recollections of Software Pioneers, Wiley, November 1997. *An excellent historical piece by one of the pioneers of virtual memory and working sets.*

[G+95] "Idleness is not sloth" by Richard Golding, Peter Bosch, Carl Staelin, Tim Sullivan, John Wilkes. USENIX ATC '95, New Orleans, Louisiana. *A fun and easy-to-read discussion of how idle time can be better used in systems, with lots of good examples.*

[LL82] "Virtual Memory Management in the VAX/VMS Operating System" by Hank Levy, P. Lipman. IEEE Computer, Vol. 15, No. 3, March 1982. *Not the first place where page clustering was used, but a clear and simple explanation of how such a mechanism works. We sure cite this paper a lot!*

 © 2008–23, ARPACI-DUSSEAU

作业（测量）

这个作业会向你介绍一个新工具 **vmstat**，以及如何用它来了解内存、CPU 和 I/O 的使用情况。在做下面的练习和问题之前，请先阅读相关的 README，并看看 `mem.c` 中的代码。

问题

1. 首先，在同一台机器上打开两个独立的终端连接，这样你就可以方便地在一个窗口里运行一个东西，在另一个窗口里运行另一个。
   现在，在一个窗口中运行 `vmstat 1`，它会每秒显示一次机器使用情况的统计信息。阅读手册页（manual page）、相关的 README 以及你需要的其他资料，弄懂它的输出。在下面的所有练习中，都让这个窗口一直运行 `vmstat`。
   接下来，我们运行程序 `mem.c`，但只使用很少的内存。输入 `./mem 1` 就可以做到（它只使用 1 MB 内存）。运行 `mem` 时，CPU 使用情况的统计数据有什么变化？`user` 时间这一列的数字合理吗？同时运行多个 `mem` 实例时，情况又会怎样变化？

2. 现在，我们一边运行 `mem`，一边观察一些内存统计数据。我们重点关注两列：`swpd` 和 `free`，分别表示已使用的虚拟内存量和空闲内存量。运行 `./mem 1024`（它会分配 1024 MB），观察这些值如何变化。然后终止正在运行的程序（按 control-c），再观察这些值如何变化。你注意到这些值有什么特点？特别是，程序退出时，`free` 这一列有什么变化？`mem` 退出时，空闲内存的增加量是否符合预期？

3. 接下来我们看交换相关的两列（`si` 和 `so`），它们表示有多少数据从磁盘换入、又有多少换出到磁盘。当然，要让这两列有动静，你需要用大量内存来运行 `mem`。首先，看看你的 Linux 系统上有多少空闲内存（比如输入 `cat /proc/meminfo`；输入 `man proc` 可以了解 `/proc` 文件系统的细节，以及在那里能找到哪些类型的信息）。`/proc/meminfo` 中最前面的几项之一，就是系统的内存总量。假设大约是 8 GB；那么先运行 `mem 4000`（大约 4 GB），观察换入/换出这两列。它们会出现非零值吗？然后再试 5000、6000 等。与第一轮循环相比，程序进入第二轮循环（以及之后的循环）时，这些值有什么变化？在第二轮、第三轮以及之后的循环中，（总共）有多少数据被换入和换出？（这些数字合理吗？）

4. 做和上面相同的实验，但这次观察其他统计数据，比如 CPU 利用率（utilization）和块 I/O 统计。运行 `mem` 时，它们有什么变化？

5. 现在我们来看看性能。为 `mem` 选一个能轻松放进内存的输入（比如系统内存为 8 GB 时选 4000）。第 0 轮循环（以及之后的第 1、2 轮等）要多长时间？然后选一个明显超出内存大小的值（同样假设内存为 8 GB，比如选 12000）。这时每轮循环要多长时间？带宽（bandwidth）数字相比如何？不停交换时的性能，和所有数据都能轻松放进内存时相比，差别有多大？你能画一张图，以 `mem` 使用的内存大小为 x 轴、访问这些内存的带宽为 y 轴吗？最后，在全部能放进内存和放不进内存这两种情况下，第一轮循环的性能与之后的循环相比如何？

 © 2008–23, ARPACI-DUSSEAU

6. 交换空间不是无限的。你可以用带 `-s` 选项的 `swapon` 工具查看有多少交换空间可用。如果用越来越大的值运行 `mem`，直到超出看起来可用的交换空间，会发生什么？内存分配在什么时候会失败？
7. 最后，如果你水平比较高，可以用 `swapon` 和 `swapoff` 配置系统使用不同的交换设备。详情请阅读手册页。如果你能用上不同的硬件，看看交换到传统硬盘、基于闪存的 SSD，甚至 RAID 阵列时，交换性能有什么变化。用更新的设备能把交换性能提高多少？能多接近在内存中运行的性能？

 © 2008–23, ARPACI-DUSSEAU